МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена Трудового Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования

МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

ОТЧЁТ

по лабораторной работе № 3

«Многотабличные запросы и аналитические представления»

по дисциплине «Математические основы баз данных»

Тема сквозного кейса: «Цифровая платформа торговли кибернетическими имплантами и био-модификациями “КИиБ”»

Выполнил студент группы БВТ2505

Гущин Михаил Борисович

Проверил: Быватов Семён Юрьевич

Москва 2026

1. ОБЩИЕ СВЕДЕНИЯ

Лабораторная работа выполнена на базе kiib, созданной в лабораторной работе №1 (схема из 10 таблиц) и наполненной в лабораторной работе №2 (1000 товаров, 1200 заказов и т.д.). Ничего заново не наполнялось — все запросы читающие, дополнительно созданы два представления (VIEW).

Задачи работы: запрос с INNER JOIN трёх таблиц; запрос с LEFT JOIN, показывающий строки без соответствий; группировка с агрегатными функциями; фильтрация групп через HAVING; по одному запросу с подзапросом каждого вида (скалярный, IN, EXISTS); обычный и рекурсивный CTE; два представления для типовых отчётов.

2. INNER JOIN ТРЁХ ТАБЛИЦ

Товар соединяется с категорией (по category_id) и продавцом (по seller_id) одновременно — в выборке видно название категории и имя продавца вместо их числовых id.

```
SELECT
    p.title,
    c.name  AS category,
    s.name  AS seller,
    p.price
FROM products p
JOIN categories c ON c.id = p.category_id
JOIN sellers    s ON s.id = p.seller_id
ORDER BY p.price DESC
LIMIT 20;
```

Query Query History

```
-- ============================================================
-- 3.1 INNER JOIN трёх таблиц: товар — категория — продавец
-- ============================================================

SELECT
    p.title,
    c.name  AS category,
    s.name  AS seller,
    p.price
FROM products p
JOIN categories c ON c.id = p.category_id
JOIN sellers    s ON s.id = p.seller_id
ORDER BY p.price DESC
LIMIT 20;
```

Data Output Messages Notifications

	title text	category text	seller text	price numeric (12,2)
1	Sandevistan Mk.4	Нейроинтерфейсы	Militech Tactical Augments	45000.00
2	Mantis Blades	Конечности	Arasaka Cyberware Division	38000.00
3	Cyberdeck Tetratronic Rippers...	Кибердеки	Zetatech Neural Systems	32000.00
4	Gorilla Arms	Конечности	Militech Tactical Augments	27500.00
5	Дермал-слой Стандарт Mk.4	Дермальные покрыт...	Viktor's Clinic (частный рипер-д...	24781.53
6	Кибер-модуль Спорт Mk.3	Конечности	Kiroshi Optics	24767.01
7	Кибер-модуль Профи Mk.3	Мышечные каркасы	Arasaka Cyberware Division	24732.11
8	Оптик-чип Спорт Mk.4	Мышечные каркасы	Arasaka Cyberware Division	24695.56
9	Экзоскелет Спорт Mk.4	Дермальные покрыт...	Arasaka Cyberware Division	24693.63
10	Кровяной фильтр Стелс Mk.4	Оптические системы	Viktor's Clinic (частный рипер-д...	24676.50
11	Кровяной фильтр Спорт Mk.5	Кибердеки	Arasaka Cyberware Division	24616.33
12	Нейролинк Спорт Mk.2	Конечности	Arasaka Cyberware Division	24594.83
13	Кибер-модуль Милитари Mk.4	Кибердеки	Kiroshi Optics	24586.39
14	Кибер-модуль Люкс Mk.2	Нейроинтерфейсы	Biotechnica Organic Solutions	24552.07
15	Реактор Милитари Mk.2	Дермальные покрыт...	Biotechnica Organic Solutions	24545.77
16	Нейролинк Милитари Mk.1	Кровеносная система	Biotechnica Organic Solutions	24512.33
17	Дермал-слой Милитари Mk.1	Кровеносная система	Kang Tao Combat Systems	24495.13
18	Кровяной фильтр Стелс Mk.3	Мышечные каркасы	Zetatech Neural Systems	24493.98
19	Нейролинк Стандарт Mk.4	Оптические системы	Zetatech Neural Systems	24491.23
20	Реактор Спорт Mk.2	Дермальные покрыт...	Militech Tactical Augments	24445.18

3. LEFT JOIN: СТРОКИ БЕЗ СООТВЕТСТВИЙ

LEFT JOIN сохраняет все строки products, даже если для товара нет ни одной связанной строки в reviews — в этом случае столбцы правой таблицы заполняются NULL. Условие WHERE r.id IS NULL отбирает именно «несовпавшие» строки — товары, которые никто не оценил.

```
SELECT p.title, p.price
FROM products p
LEFT JOIN reviews r ON r.product_id = p.id
WHERE r.id IS NULL
ORDER BY p.price DESC
LIMIT 20;
```

Query | Query History

```sql
-- ============================================================
-- 3.2 LEFT JOIN: товары без единого отзыва
-- ============================================================

SELECT p.title, p.price
FROM products p
LEFT JOIN reviews r ON r.product_id = p.id
WHERE r.id IS NULL
ORDER BY p.price DESC
LIMIT 20;
```

Data Output | Messages | Notifications

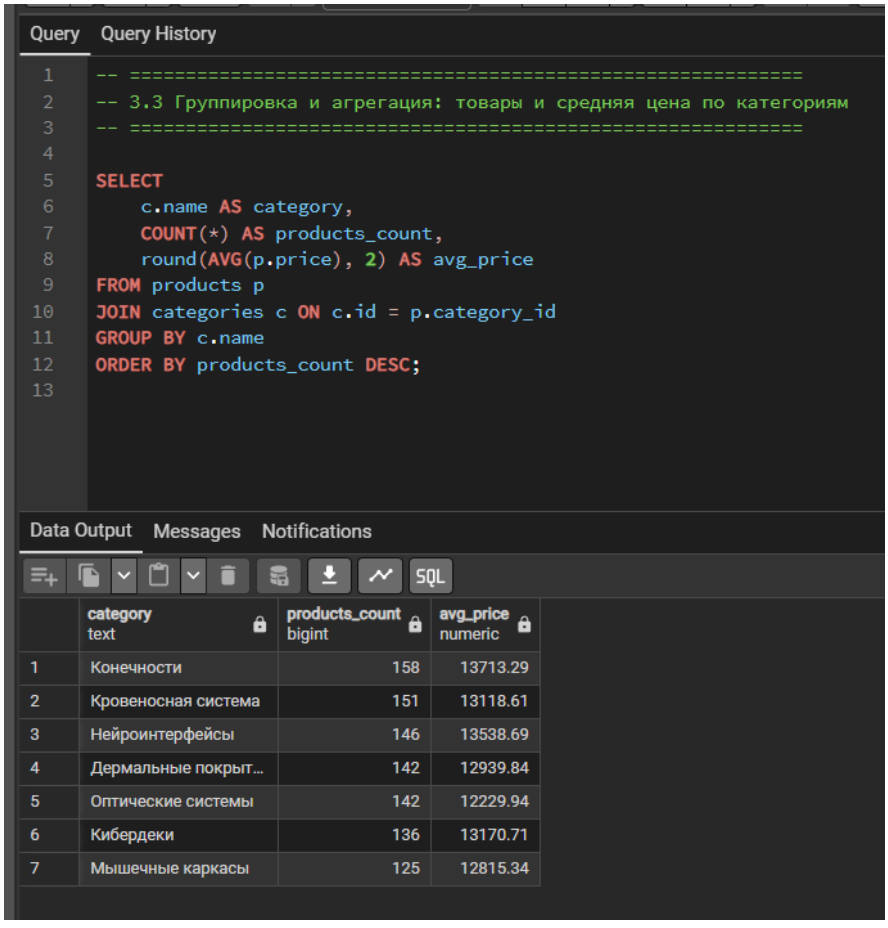
SQL

	title text	price numeric (12,2)
1	Дермал-слой Стандарт Mk.4	24781.53
2	Кибер-модуль Спорт Mk.3	24767.01
3	Экзоскелет Спорт Mk.4	24693.63
4	Кибер-модуль Милитари Mk.4	24586.39
5	Дермал-слой Стандарт Mk.3	24410.98
6	Кибер-модуль Профи Mk.2	24388.18
7	Нейролинк Профи Mk.5	24209.29
8	Экзоскелет Спорт Mk.4	24137.51
9	Экзоскелет Спорт Mk.3	24074.88
10	Кибер-модуль Стандарт Mk.2	24010.82
11	Нейролинк Стелс Mk.2	23911.17
12	Кибер-модуль Спорт Mk.1	23817.67
13	Экзоскелет Спорт Mk.4	23810.12
14	Кибер-модуль Милитари Mk.3	23751.69
15	Оптик-чип Стандарт Mk.4	23738.93
16	Кибер-модуль Спорт Mk.4	23703.91
17	Экзоскелет Стандарт Mk.2	23608.66
18	Дермал-слой Спорт Mk.1	23602.27
19	Мышечный каркас Стелс M...	23580.38
20	Нейролинк Спорт Mk.5	23537.47

4. ГРУППИРОВКА И АГРЕГАТНЫЕ ФУНКЦИИ

Для каждой категории считается количество товаров (COUNT) и средняя цена (AVG, округлённая до двух знаков). GROUP BY делит строки products на группы по category_id, агрегатные функции сворачивают каждую группу в одну строку результата.

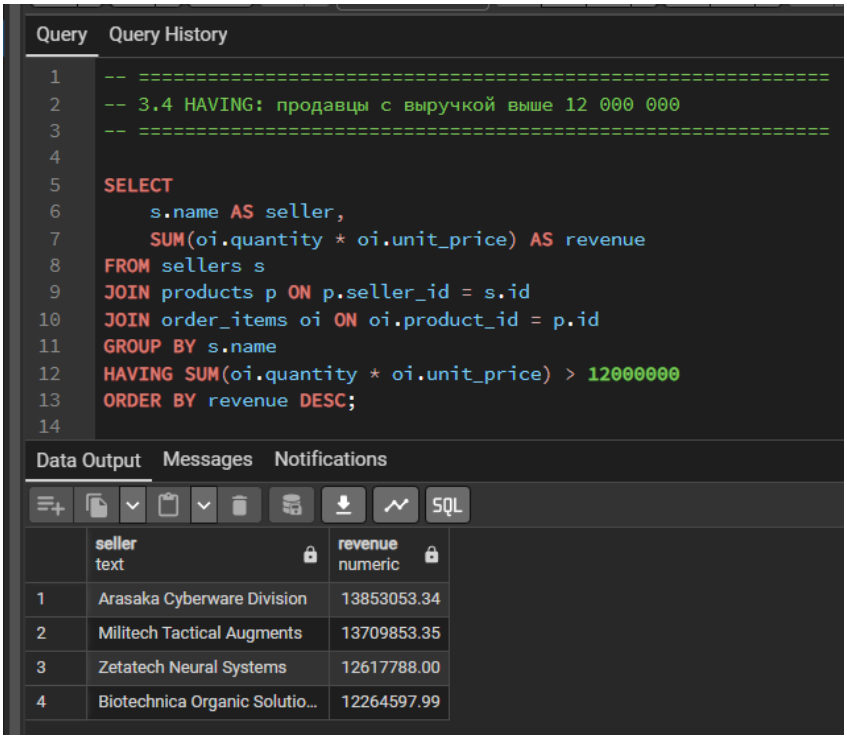
```
SELECT
    c.name AS category,
    COUNT(*) AS products_count,
    round(AVG(p.price), 2) AS avg_price
FROM products p
JOIN categories c ON c.id = p.category_id
GROUP BY c.name
ORDER BY products_count DESC;
```



5. HAVING: ФИЛЬТРАЦИЯ ГРУПП

Порог 12 000 000 подобран не произвольно, а по факту: сначала была посчитана фактическая выручка каждого продавца, после чего выбрано значение, которое реально отсекает часть продавцов (из 8 в выборку HAVING прошли не все). WHERE здесь использовать нельзя — на момент его выполнения агрегат SUM(...) ещё не посчитан, фильтровать по нему можно только после группировки, то есть в HAVING.

```
SELECT
    s.name AS seller,
    SUM(oi.quantity * oi.unit_price) AS revenue
FROM sellers s
JOIN products p ON p.seller_id = s.id
JOIN order_items oi ON oi.product_id = p.id
GROUP BY s.name
HAVING SUM(oi.quantity * oi.unit_price) > 12000000
ORDER BY revenue DESC;
```

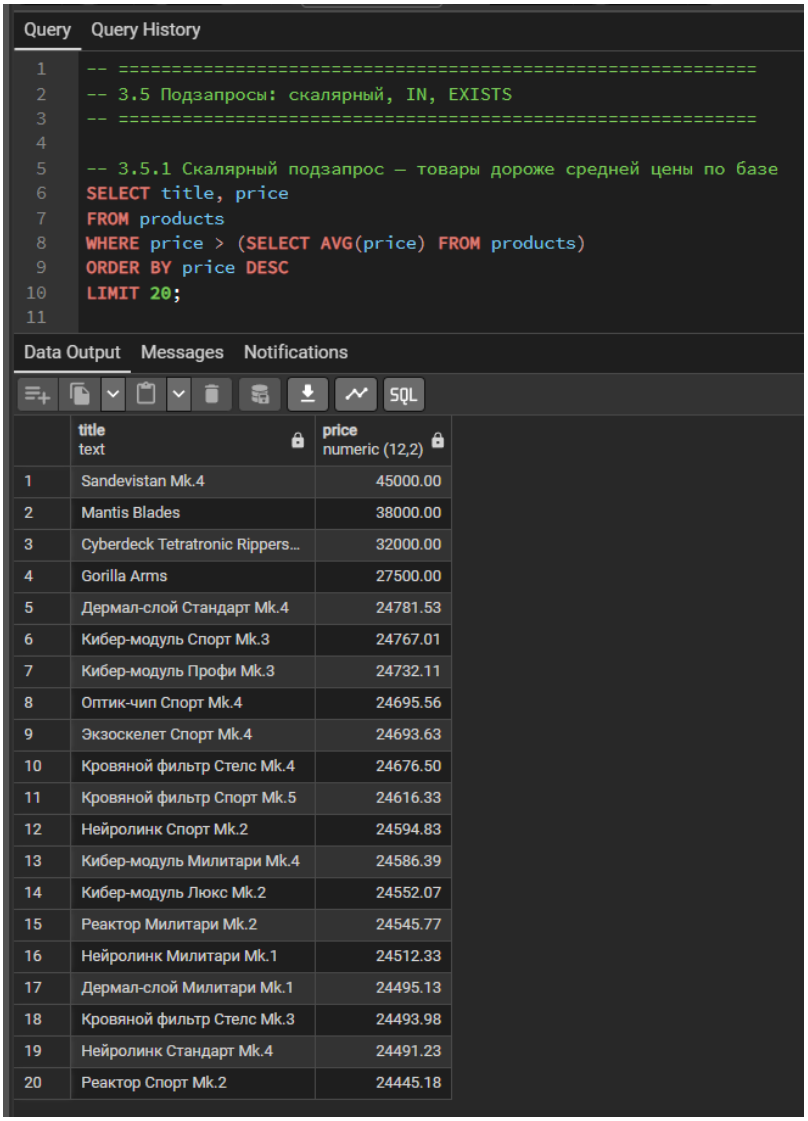


6. ПОДЗАПРОСЫ

6.1 Скалярный подзапрос

Подзапрос (SELECT AVG(price) FROM products) возвращает ровно одно число — среднюю цену по всей таблице, — и это число сравнивается с ценой каждого товара во внешнем запросе.

```
SELECT title, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC
LIMIT 20;
```



Query | Query History

```
1  -- ============================================================
2  -- 3.5 Подзапросы: скалярный, IN, EXISTS
3  -- ============================================================
4
5  -- 3.5.1 Скалярный подзапрос — товары дороже средней цены по базе
6  SELECT title, price
7  FROM products
8  WHERE price > (SELECT AVG(price) FROM products)
9  ORDER BY price DESC
10 LIMIT 20;
11
```

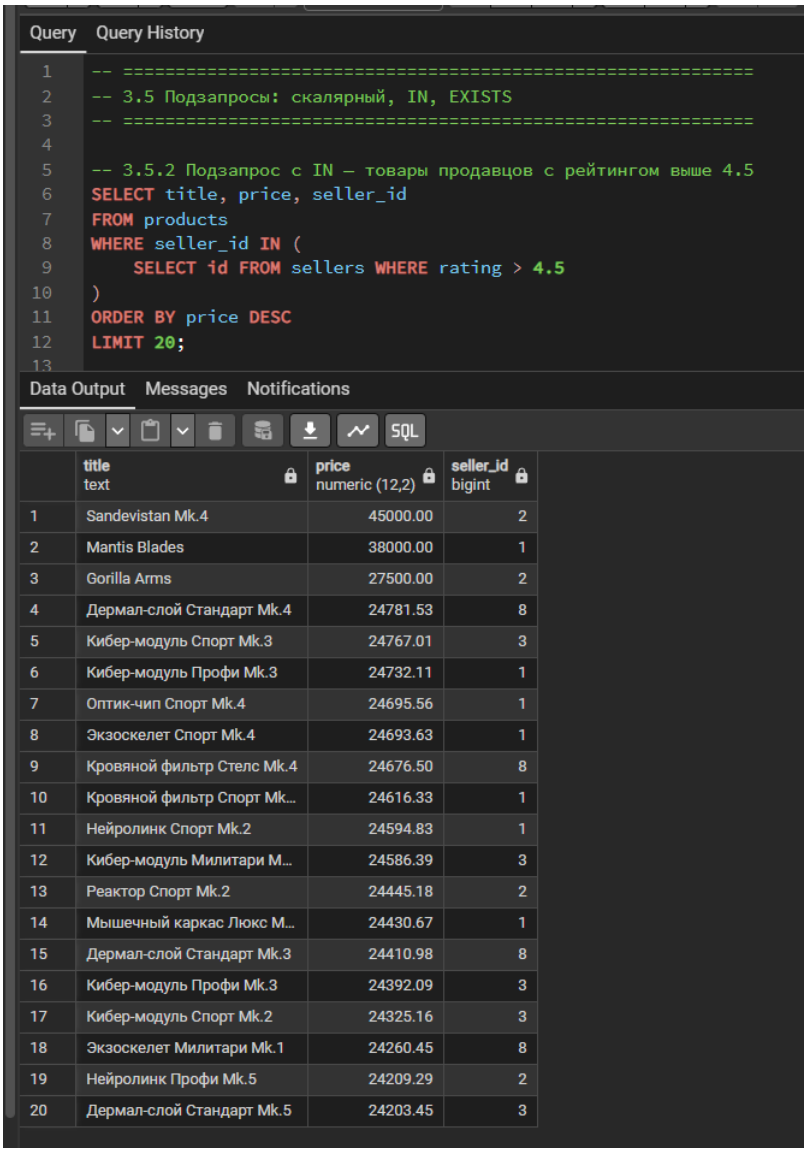
Data Output | Messages | Notifications

	title text	price numeric (12,2)
1	Sandevistan Mk.4	45000.00
2	Mantis Blades	38000.00
3	Cyberdeck Tetratronic Rippers...	32000.00
4	Gorilla Arms	27500.00
5	Дермал-слой Стандарт Mk.4	24781.53
6	Кибер-модуль Спорт Mk.3	24767.01
7	Кибер-модуль Профи Mk.3	24732.11
8	Оптик-чип Спорт Mk.4	24695.56
9	Экзоскелет Спорт Mk.4	24693.63
10	Кровяной фильтр Стелс Mk.4	24676.50
11	Кровяной фильтр Спорт Mk.5	24616.33
12	Нейролинк Спорт Mk.2	24594.83
13	Кибер-модуль Милитари Mk.4	24586.39
14	Кибер-модуль Люкс Mk.2	24552.07
15	Реактор Милитари Mk.2	24545.77
16	Нейролинк Милитари Mk.1	24512.33
17	Дермал-слой Милитари Mk.1	24495.13
18	Кровяной фильтр Стелс Mk.3	24493.98
19	Нейролинк Стандарт Mk.4	24491.23
20	Реактор Спорт Mk.2	24445.18

6.2 Подзапрос с IN

Подзапрос возвращает множество id продавцов с рейтингом выше 4.5, внешний запрос отбирает товары, чей seller_id входит в это множество.

```
SELECT title, price, seller_id
FROM products
WHERE seller_id IN (
    SELECT id FROM sellers WHERE rating > 4.5
)
ORDER BY price DESC
LIMIT 20;
```



Query | Query History

```
1  -- ============================================================
2  -- 3.5 Подзапросы: скалярный, IN, EXISTS
3  -- ============================================================
4
5  -- 3.5.2 Подзапрос с IN — товары продавцов с рейтингом выше 4.5
6  SELECT title, price, seller_id
7  FROM products
8  WHERE seller_id IN (
9      SELECT id FROM sellers WHERE rating > 4.5
10 )
11 ORDER BY price DESC
12 LIMIT 20;
13
```

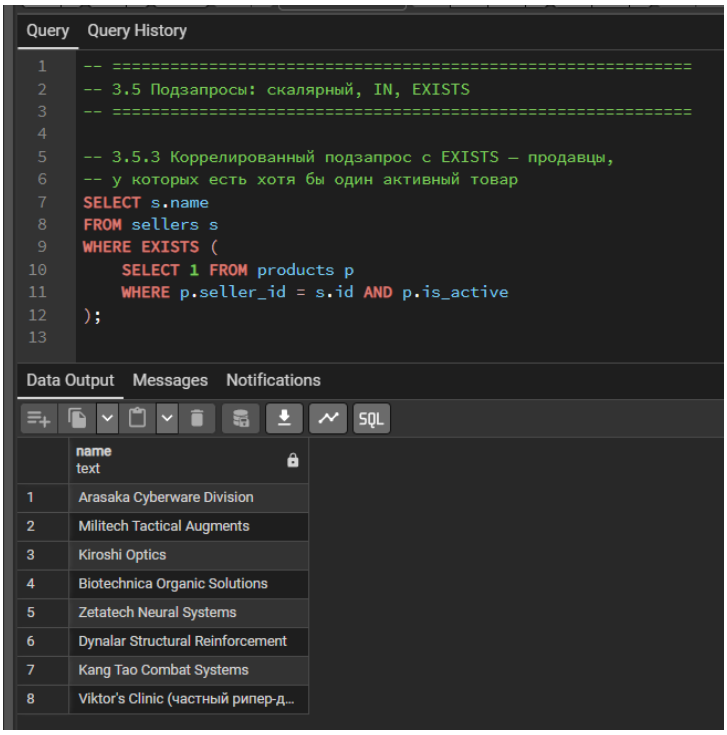
Data Output | Messages | Notifications

	title text	price numeric (12,2)	seller_id bigint
1	Sandevistan Mk.4	45000.00	2
2	Mantis Blades	38000.00	1
3	Gorilla Arms	27500.00	2
4	Дермал-слой Стандарт Mk.4	24781.53	8
5	Кибер-модуль Спорт Mk.3	24767.01	3
6	Кибер-модуль Профи Mk.3	24732.11	1
7	Оптик-чип Спорт Mk.4	24695.56	1
8	Экзоскелет Спорт Mk.4	24693.63	1
9	Кровяной фильтр Стелс Mk.4	24676.50	8
10	Кровяной фильтр Спорт Mk...	24616.33	1
11	Нейролинк Спорт Mk.2	24594.83	1
12	Кибер-модуль Милитари М...	24586.39	3
13	Реактор Спорт Mk.2	24445.18	2
14	Мышечный каркас Люкс М...	24430.67	1
15	Дермал-слой Стандарт Mk.3	24410.98	8
16	Кибер-модуль Профи Mk.3	24392.09	3
17	Кибер-модуль Спорт Mk.2	24325.16	3
18	Экзоскелет Милитари Mk.1	24260.45	8
19	Нейролинк Профи Mk.5	24209.29	2
20	Дермал-слой Стандарт Mk.5	24203.45	3

6.3 Коррелированный подзапрос с EXISTS

В отличие от двух предыдущих, этот подзапрос ссылается на внешнюю строку (p.seller_id = s.id) — то есть выполняется заново для каждого продавца. EXISTS проверяет сам факт существования хотя бы одной подходящей строки, не считая их количество — это обычно быстрее, чем IN с большим подзапросом.

```
SELECT s.name
FROM sellers s
WHERE EXISTS (
    SELECT 1 FROM products p
    WHERE p.seller_id = s.id AND p.is_active
);
```



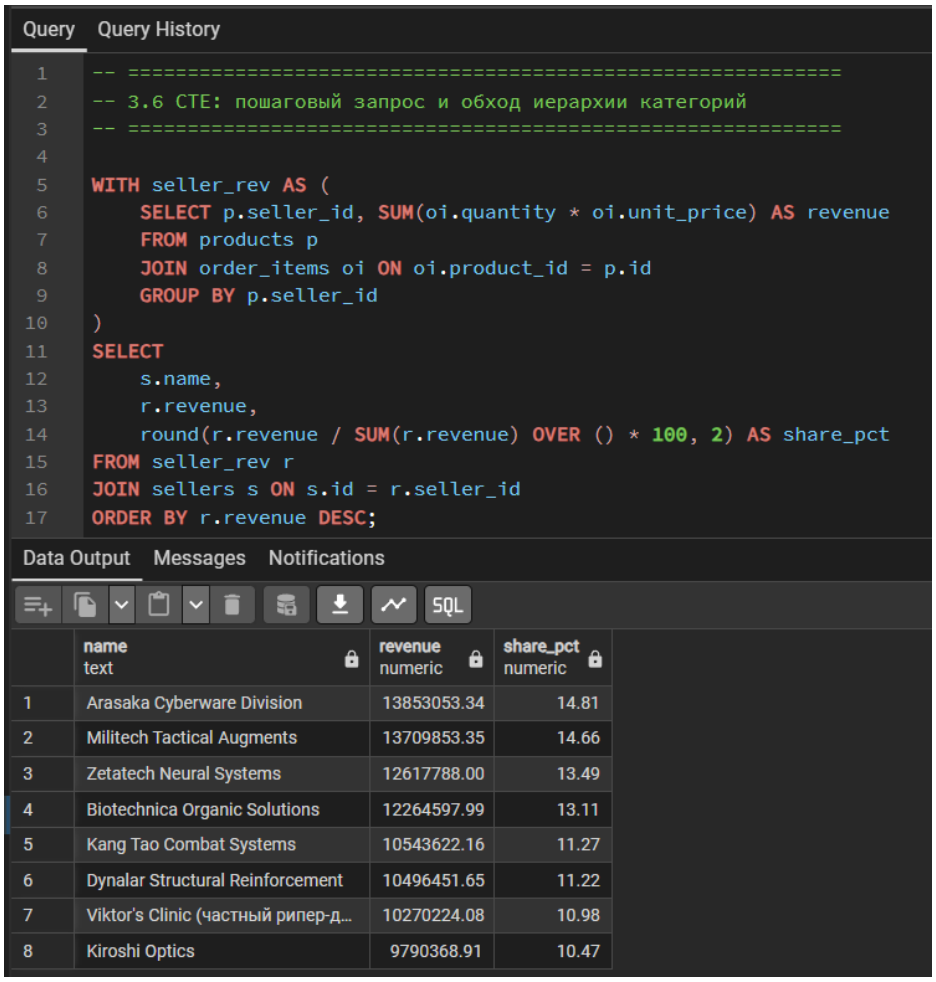
Результат содержит все 8 продавцов — у каждого есть хотя бы один активный товар.

7. CTE И РЕКУРСИВНЫЙ CTE

7.1 Обычный CTE: выручка продавца и её доля

Запрос разбит на два шага вместо одного большого выражения с вложенными агрегатами. Шаг 1 (CTE seller_rev) — считает суммарную выручку по каждому продавцу. Шаг 2 — присоединяет имя продавца и вычисляет долю каждого в общей выручке оконной функцией SUM(...) OVER ().

```
WITH seller_rev AS (
    SELECT p.seller_id, SUM(oi.quantity * oi.unit_price) AS revenue
    FROM products p
    JOIN order_items oi ON oi.product_id = p.id
    GROUP BY p.seller_id
)
SELECT
    s.name,
    r.revenue,
    round(r.revenue / SUM(r.revenue) OVER () * 100, 2) AS share_pct
FROM seller_rev r
JOIN sellers s ON s.id = r.seller_id
ORDER BY r.revenue DESC;
```



	name text	revenue numeric	share_pct numeric
1	Arasaka Cyberware Division	13853053.34	14.81
2	Militech Tactical Augments	13709853.35	14.66
3	Zetatech Neural Systems	12617788.00	13.49
4	Biotechnica Organic Solutions	12264597.99	13.11
5	Kang Tao Combat Systems	10543622.16	11.27
6	Dynalar Structural Reinforcement	10496451.65	11.22
7	Viktor's Clinic (частный рипер-д...	10270224.08	10.98
8	Kiroshi Optics	9790368.91	10.47

7.2 Рекурсивный CTE: обход дерева категорий

Якорная часть (до UNION ALL) выбирает корневые категории — те, у кого parent_id IS NULL, уровень вложенности для них 0. Рекурсивная часть (после UNION ALL) на каждом шаге находит детей уже найденных на предыдущем шаге категорий (JOIN category_tree ct ON c.parent_id = ct.id), увеличивает depth на 1 и достраивает текстовый путь от корня через конкатенацию строк. Рекурсия останавливается сама, когда на очередном шаге не находится ни одной новой дочерней категории.

```
WITH RECURSIVE category_tree AS (
    SELECT id, parent_id, name, 0 AS depth, name::text AS path
    FROM categories
    WHERE parent_id IS NULL

    UNION ALL

    SELECT c.id, c.parent_id, c.name, ct.depth + 1,
           ct.path || ' → ' || c.name
    FROM categories c
    JOIN category_tree ct ON c.parent_id = ct.id
)
SELECT id, depth, path
FROM category_tree
ORDER BY path;
```

Query | Query History

```sql
-- ============================================================
-- 3.6 CTE: пошаговый запрос и обход иерархии категорий
-- ============================================================

-- 3.6.2 Рекурсивный CTE — обход всего дерева категорий с указанием
-- глубины вложенности и полного пути от корня
WITH RECURSIVE category_tree AS (
    -- якорная часть: корневые категории (parent_id IS NULL)
    SELECT id, parent_id, name, 0 AS depth, name::text AS path
    FROM categories
    WHERE parent_id IS NULL

    UNION ALL

    -- рекурсивная часть: находим детей уже найденных категорий
    SELECT c.id, c.parent_id, c.name, ct.depth + 1, ct.path || ' → ' || c.name
    FROM categories c
    JOIN category_tree ct ON c.parent_id = ct.id
)
SELECT id, depth, path
FROM category_tree
ORDER BY path;
```

Data Output | Messages | Notifications

	id bigint	depth integer	path text
1	2	0	Био-модификации
2	9	1	Био-модификации → Дермальные покрытия
3	11	1	Био-модификации → Кровеносная система
4	10	1	Био-модификации → Мышечные каркасы
5	1	0	Кибернетические импланты
6	8	1	Кибернетические импланты → Кибердеки
7	7	1	Кибернетические импланты → Конечности
8	6	1	Кибернетические импланты → Нейроинтерфейсы
9	5	1	Кибернетические импланты → Оптические систе...
10	3	0	Программное обеспечение
11	4	0	Экипировка рипер-дока

Результат — все 11 категорий с указанием глубины вложенности (0 — корневые, 1 — дочерние) и полного пути от корня. Видно, что у «Программного обеспечения» и «Экипировки рипер-дока» детей нет — рекурсия для них останавливается сразу на якорной строке.

8. ПРЕДСТАВЛЕНИЯ

8.1 seller_revenue_report — отчёт по продавцам

LEFT JOIN (а не INNER) использован намеренно: если бы у какого-то продавца не было ни одного товара или ни одной продажи, INNER JOIN просто не включил бы его в отчёт. LEFT JOIN сохраняет строку продавца в любом случае, а COALESCE(SUM(...), 0) подставляет 0 вместо NULL там, где продаж не было.

```
CREATE OR REPLACE VIEW seller_revenue_report AS
SELECT
    s.id AS seller_id,
    s.name AS seller,
    COUNT(DISTINCT p.id) AS products_count,
    COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS total_revenue
FROM sellers s
LEFT JOIN products p ON p.seller_id = s.id
LEFT JOIN order_items oi ON oi.product_id = p.id
GROUP BY s.id, s.name;

SELECT * FROM seller_revenue_report ORDER BY total_revenue DESC;
```

Query | Query History

```
-- ============================================================
-- 3.7 Представления для типовых отчётов
-- ============================================================

-- Отчёт по продавцам: сколько товаров, какая суммарная выручка.
-- LEFT JOIN — чтобы в отчёте остались и продавцы без продаж
-- (выручка у них будет 0, а не пропавшая строка).
CREATE OR REPLACE VIEW seller_revenue_report AS
SELECT
    s.id AS seller_id,
    s.name AS seller,
    COUNT(DISTINCT p.id) AS products_count,
    COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS total_revenue
FROM sellers s
LEFT JOIN products p ON p.seller_id = s.id
LEFT JOIN order_items oi ON oi.product_id = p.id
GROUP BY s.id, s.name;


-- обращение к представлениям — используются как обычные таблицы
SELECT * FROM seller_revenue_report ORDER BY total_revenue DESC;
```

Data Output | Messages | Notifications

	seller_id bigint	seller text	products_count bigint	total_revenue numeric
1	1	Arasaka Cyberware Division	153	13853053.34
2	2	Militech Tactical Augments	131	13709853.35
3	5	Zetatech Neural Systems	123	12617788.00
4	4	Biotechnica Organic Solutions	122	12264597.99
5	7	Kang Tao Combat Systems	110	10543622.16
6	6	Dynalar Structural Reinforcement	125	10496451.65
7	8	Viktor's Clinic (частный рипер-д...	111	10270224.08
8	3	Kiroshi Optics	125	9790368.91

8.2 product_catalog_view — каталог товаров

Представление собирает «витринный» вид товара: вместо числовых category_id и seller_id — читаемые названия, плюс остаток на складе из stock (через LEFT JOIN — товар должен быть виден в каталоге, даже если строка в stock почему-то отсутствует). Обращение к представлению ничем не отличается от обращения к обычной таблице — это видно в примере с фильтром по неактивным товарам.

```
CREATE OR REPLACE VIEW product_catalog_view AS
SELECT
    p.id,
    p.title,
    c.name AS category,
    s.name AS seller,
    p.price,
    p.is_active,
    st.quantity AS stock_quantity
FROM products p
JOIN categories c ON c.id = p.category_id
JOIN sellers s ON s.id = p.seller_id
LEFT JOIN stock st ON st.product_id = p.id;

SELECT * FROM product_catalog_view
WHERE is_active = FALSE
ORDER BY price DESC
LIMIT 20;
```

Query | Query History

```sql

-- Каталог товаров с «человеческими» названиями категории и
-- продавца вместо id, плюс остаток на складе — то, что обычно
-- показывают на витрине маркетплейса.
CREATE OR REPLACE VIEW product_catalog_view AS
SELECT
    p.id,
    p.title,
    c.name AS category,
    s.name AS seller,
    p.price,
    p.is_active,
    st.quantity AS stock_quantity
FROM products p
JOIN categories c ON c.id = p.category_id
JOIN sellers s ON s.id = p.seller_id
LEFT JOIN stock st ON st.product_id = p.id;


-- обращение к представлениям — используются как обычные таблицы
SELECT * FROM product_catalog_view
WHERE is_active = FALSE
ORDER BY price DESC
LIMIT 20;
```

Data Output | Messages | Notifications

SQL

	id bigint	title text	category text	seller text	price numeric (12,2)	is_active boolean	stock_quantity integer
1	275	Экзоскелет Спорт Mk.4	Дермальные покры…	Arasaka Cyberware Division	24693.63	false	124
2	863	Нейролинк Милитари Mk.1	Кровеносная систе…	Biotechnica Organic Solutions	24512.33	false	50
3	294	Дермал-слой Стандарт Mk.3	Мышечные каркасы	Viktor's Clinic (частный ripper-д…	24410.98	false	57
4	15	Реактор Стандарт Mk.5	Кибердеки	Viktor's Clinic (частный ripper-д…	24157.75	false	38
5	552	Кибер-модуль Стандарт Mk.2	Дермальные покры…	Viktor's Clinic (частный ripper-д…	24010.82	false	86
6	148	Кибер-модуль Спорт Mk.1	Кровеносная систе…	Kiroshi Optics	23817.67	false	100
7	683	Экзоскелет Спорт Mk.4	Кровеносная систе…	Kang Tao Combat Systems	23810.12	false	204
8	830	Оптик-чип Милитари Mk.1	Кибердеки	Militech Tactical Augments	23465.04	false	102
9	189	Кибер-модуль Люкс Mk.5	Конечности	Arasaka Cyberware Division	23443.45	false	196
10	957	Дермал-слой Профи Mk.4	Кровеносная систе…	Biotechnica Organic Solutions	23344.88	false	117
11	662	Кибер-модуль Профи Mk.1	Нейроинтерфейсы	Dynalar Structural Reinforcement	23279.72	false	198
12	133	Оптик-чип Спорт Mk.3	Конечности	Kiroshi Optics	23232.99	false	141
13	862	Кровяной фильтр Стелс Mk.2	Нейроинтерфейсы	Viktor's Clinic (частный ripper-д…	22968.17	false	4
14	270	Мышечный каркас Стандарт Mk.4	Конечности	Viktor's Clinic (частный ripper-д…	22790.14	false	195
15	246	Экзоскелет Спорт Mk.2	Кровеносная систе…	Kang Tao Combat Systems	22745.03	false	238
16	147	Оптик-чип Стелс Mk.2	Кровеносная систе…	Viktor's Clinic (частный ripper-д…	22045.23	false	13
17	401	Кибер-модуль Стандарт Mk.5	Кровеносная систе…	Arasaka Cyberware Division	21211.49	false	155
18	869	Мышечный каркас Милитари M…	Кровеносная систе…	Kiroshi Optics	20851.92	false	204
19	772	Экзоскелет Стелс Mk.4	Конечности	Kiroshi Optics	20677.52	false	46
20	885	Нейролинк Люкс Mk.5	Оптические системы	Viktor's Clinic (частный ripper-д…	20470.38	false	190

9. ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы №3 на базе kiib (схема из лаб.1, данные из лаб.2):

— написан запрос с INNER JOIN трёх таблиц (товар — категория — продавец);

— написан запрос с LEFT JOIN, отбирающий строки без соответствий в связанной таблице (товары без отзывов);

— построен запрос с группировкой и агрегатными функциями COUNT/AVG по категориям;

— применено HAVING для фильтрации групп по выручке продавца;

— написано по одному запросу с подзапросом каждого вида — скалярным, с IN и коррелированным с EXISTS;

— использован обычный CTE для разбиения запроса на шаги и рекурсивный CTE для обхода дерева категорий с вычислением глубины вложенности;

— созданы два представления (seller_revenue_report, product_catalog_view) для типовых отчётов платформы и продемонстрировано обращение к ним как к обычным таблицам.